

位元計算(佔分 20 分)

問題描述：

微處理器的暫存器(Register)是一種超高速暫存記憶單元，用於暫時存放運算資料、指令或控制週邊設備的狀態。暫存器長度大多為 8-、16- 或 32-bit，每個位元通常對應某個硬體功能或狀態。程式開發者必須熟捻讀取或修改暫存器中某個位元以實現對於系統的控制。試撰寫程式去分別讀取 2 個 16-bit 整數 v1、v2 和位元運算模式 m，最後將 v1 和 v2 經過指定的位元運算後印出結果 8-bit 整數 n，其中位元運算模式共分成 4 種運算方式，分別敘述如下：

運算模式 a：將 v1 的奇數位元與 v2 的偶數位元進行 OR 運算。

運算模式 b：將 v1 的偶數位元與 v2 的奇數位元進行 XOR 運算。

運算模式 c：將 v1 的奇數位元經過反向運算後與 v2 的奇數位元進行 AND 運算。

運算模式 d：分別將 v1 和 v2 本身內的位元內容進行交換：如第 15 與第 8 位元互換、第 14 與第 9 位元互換、第 13 與第 10 互換、第 12 與第 11 位元互換、第 7 位元與第 0 位元互換、第 6 位元與第 1 位元互換、第 5 位元與第 2 位元互換、第 4 位元與第 3 位元互換。將互換位元後的 v1 和 v2 相加後，最後將其高位元組內容和低位元組內容進行 XOR 運算後輸出。

例如讀取 v1 整數為 220、v2 整數為 50090、運算模式 m 為 a 時，則 v1 整數的位元內容依序為 0000000011011100、v2 整數的位元內容為 1100001110101010，則 v1 的奇數位元依序為 00001010、v2 的偶數位元分別為 10010000，將其 OR 運算後得出 10011111，轉換整數為 154。

輸入說明：

第 1 行是為 k 筆資料需要經過位元運算，其中 $1 \leq k \leq 100$ 。

第 2 行至第 k 行資料分別為 16-bit 整數 v1、v2 和運算模式 m，其中 $0 \leq \{v1, v2\} \leq 65535$ ， $m \in \{a, b, c, d\}$ 。

輸出說明：

v1 和 v2 將根據位元運算模式 m，經計算後印出 8-bit 整數內容 n，其中 $0 \leq n < 255$ 。

範例 1：

輸入：

2

220 50090 a

16987 52100 c

輸出：

154

168

範例 2：

輸入：

2

11111 50000 d

20 15 b

輸出：

103

5

Bitwise Calculation (20 points)

Problem Description:

A microprocessor's register is a high-speed temporary storage unit used for storing operational data, instructions, or peripheral device states. Most registers are 8, 16, or 32 bits long, where each bit usually corresponds to a specific hardware function or status. Developers must be skilled in reading or modifying specific bits within a register to control the system.

Write a program to read two 16-bit integers v_1 , v_2 and a bitwise operation mode m . Perform the specified operation on v_1 and v_2 and output the resulting 8-bit integer n . There are four operation modes:

- **Mode a:** Extract the **odd-positioned bits** (bits 15, 13, 11, 9, 7, 5, 3, 1) from v_1 and the **even-positioned bits** (bits 14, 12, 10, 8, 6, 4, 2, 0) from v_2 . Perform an **OR** operation between these two sets of bits.
- **Mode b:** Extract the **even-positioned bits** from v_1 and the **odd-positioned bits** from v_2 . Perform an **XOR** operation between them.
- **Mode c:** Extract the **odd-positioned bits** from v_1 , **invert** them (NOT operation), and then perform an **AND** operation with the **odd-positioned bits** of v_2 .
- **Mode d:** Perform an internal bit swap for both v_1 and v_2 . In each 16-bit integer, swap bit 15 with bit 8, 14 with 9, 13 with 10, 12 with 11, and swap bit 7 with bit 0, 6 with 1, 5 with 2, 4 with 3. **Add** the two modified integers together. Finally, perform an **XOR** operation between the **high-byte** (bits 15-8) and **low-byte** (bits 7-0) of the resulting sum to produce the output.

Example: If $v_1 = 220$, $v_2 = 50090$, and mode $m = a$: v_1 (binary): 00000000 11011100, v_2 (binary): 11000011 10101010, v_1 odd bits: 00001010, v_2 even bits: 10010000 The OR result is 10011010, which equals **154** in decimal.

Input Description:

Line 1: An integer $1 \leq k \leq 100$, representing the number of cases.

Line 2 to Line $1+k$: Two 16-bit integers v_1 , v_2 and an operation mode m , where $0 \leq \{v_1, v_2\} \leq 65535$ and $m \in \{a, b, c, d\}$.

Output Description:

For each case, output the calculated 8-bit integer n ($0 \leq n < 255$).

Example 1:

Input:

2

220 50090 a

16987 52100 c

Output:

154

168

Example 2:

Input:

2

11111 50000 d

20 15 b

Output:

103

5